

Bioscoop: A Domain-Specific Language for FFmpeg Filtergraph Composition

Daniel Szmulewicz
Independent Researcher
daniel.szmulewicz@gmail.com

Abstract—We present Bioscoop, a domain-specific language compiler that targets FFmpeg filtergraph syntax. Bioscoop provides two front-ends — an external text-based DSL parsed via GLL parsing, and an internal macro-based DSL embedded in Clojure — that converge on an identical abstract syntax tree before evaluation. This architectural property, which we call *AST convergence*, is a natural consequence of Lisp homoiconicity applied deliberately, and enables a single definitional interpreter to handle both input paths. Beyond the two-front-end architecture, we describe several properties of the system that are of independent interest: an intermediate representation that is isomorphic to the target language’s grammar, eliminating the lowering problem; a first-class iteration construct that brings Turing-complete abstraction inside the filtergraph language; multi-stage programming via `&env` for threading lexical bindings into the DSL runtime; a fail-soft error discipline appropriate for interactive creative coding; and dual resolution strategies that maximize the affordances of two distinct execution environments. We situate these contributions within programming language theory and discuss the domain conditions under which the isomorphic IR approach is applicable.

Index Terms—Domain Specific Languages, FFmpeg, Filtergraph, Multi-modal DSL, Clojure, Homoiconicity, Intermediate Representation, Multi-stage Programming

I. INTRODUCTION

FFmpeg is the dominant open-source multimedia processing framework, used for video and audio encoding, transcoding, filtering, and composition. Its filtergraph syntax — a declarative language for composing processing pipelines — is expressive but static: it offers no abstraction mechanisms, no variables, no iteration, and no conditional logic. Complex filtergraphs must be written out in full or generated by string manipulation outside the framework.

This paper describes Bioscoop, a compiler from a Lisp-like domain-specific language to FFmpeg filtergraph syntax. Bioscoop addresses FFmpeg’s expressiveness gap by providing:

- Named, reusable filter graph definitions (`defgraph`)
- Lexical binding (`let`)
- A first-class iteration construct (`for`) that generates parallel filtergraph structures
- Parameterized filter arguments computed at compile time
- Spec-driven parameter validation

The system supports two syntactically distinct front-ends that share a single evaluation pipeline. We describe the

architectural consequences of this design, with particular attention to the properties that distinguish Bioscoop from string-generation approaches and from prior DSL work in the multimedia domain.

A. Motivating Example

Consider generating a slideshow that cross-fades n images. In raw FFmpeg, every intermediate label must be written explicitly. For ten images this requires approximately ninety characters of filtergraph syntax, all of which must be manually regenerated if n changes. In Bioscoop:

```
1 (defn slideshow [n]
2   (bioscoop
3     (for [i (range n)]
4       [(str i)] (loop {:loop 124 :size 1})
5         [(str "l" i)]))
6     (for [i (range 1 n)]
7       [(if (= i 1) "l0" (str "v" i))
8        (str "l" i)]
9       (xfade {:transition "fade"
10                :duration 1
11                :offset (+ (* i 4) 3)})
12       [(str "v" i)])))
```

Listing 1. Parameterized slideshow in Bioscoop

`(slideshow 10)` produces the equivalent of ninety lines of hand-written FFmpeg filtergraph syntax. Changing n , the transition type, or the timing requires changing a single argument.

II. BACKGROUND AND RELATED WORK

A. FFmpeg Filtergraph Syntax

An FFmpeg filtergraph is a semicolon-separated sequence of filterchains, each a comma-separated sequence of filters. Filters carry optional bracketed input and output link labels. The BNF grammar is:

```
1 filtergraph = filterchain ("," filterchain)*
2 filterchain = filter ("," filter)*
3 filter      = linklabel* filter-spec linklabel*
4 filter-spec = name ("=" arguments)?
5 linklabel   = "[" name "]"
```

Listing 2. FFmpeg filtergraph grammar (simplified)

The grammar is context-free and unambiguous. There are no variables, no abstraction mechanisms, and no iteration. Filtergraphs of any complexity must be written as single expressions.

B. Existing Approaches

Existing tools for programmatic FFmpeg filtergraph generation fall into two categories. *API wrappers* such as `ffmpeg-python` [1] and `fluent-ffmpeg` [2] expose FFmpeg’s options as function calls but treat the filtergraph itself as an opaque string to be concatenated. *Higher-level abstractions* such as `MoviePy` [3] hide FFmpeg almost entirely, trading expressiveness for simplicity. Neither category models the filtergraph as a structured intermediate representation.

C. DSL Implementation in Lisp

The design of internal DSLs in Lisp is well-studied. Hudak’s taxonomy [4] distinguishes shallow embeddings, where host language constructs carry DSL semantics, from deep embeddings, where DSL programs are represented as data and interpreted by a separate evaluator. Bioscoop uses a deep embedding: Clojure forms are rewritten into a tagged vector AST that is then evaluated by a definitional interpreter.

Racket’s `#lang` mechanism [5] and Clojure’s macro system both support what we call AST convergence naturally, though we are not aware of prior work that names or formalizes this property in the context of multi-modal DSLs.

III. ARCHITECTURE

A. Overview

Bioscoop consists of six layers:

- 1) *Two front-ends*: a GLL parser for the external text DSL and a macro for the internal Clojure DSL
- 2) *A shared AST*: tagged Clojure vectors produced identically by both front-ends
- 3) *A definitional interpreter*: `transform-ast`, a multimethod dispatching on AST node type
- 4) *An intermediate representation*: algebraic records (`Filter`, `FilterChain`, `FilterGraph`)
- 5) *A back-end*: `to-ffmpeg`, a serializer from IR to filtergraph string
- 6) *An error channel*: a dynamically-scoped accumulator for compile-time errors

B. AST Convergence

Both front-ends produce ASTs consisting of tagged Clojure vectors. The node types are: `:program`, `:compose`, `:for-binding`, `:let-binding`, `:binding`, `:padded-graph`, `:list`, `:map`, `:symbol`, `:keyword`, `:string`, `:number`, `:boolean`, `:label`, `:graph-definition`.

Formally, let $P_e : L_e \rightarrow AST$ be the external parser and $M_i : L_i \rightarrow AST$ be the macro. AST convergence holds when:

$$\forall p \in L_e \cap L_i : P_e(p) = M_i(p) \quad (1)$$

where $L_e \cap L_i$ denotes programs expressible in both surface syntaxes. This enables a single transformer T :

$AST \rightarrow IR$ to handle both paths, guaranteeing semantic equivalence by construction.

This property is a natural consequence of Lisp homoiconicity applied deliberately: the `form->ast` function in the macro path converts Clojure s-expressions into the same tagged vector structure the GLL parser produces. We describe it here because the deliberate design decision to keep the two ASTs identical — rather than allowing them to diverge as implementations evolve — is what makes the guarantee hold in practice.

The external front-end uses `Instaparse` [7], which implements the GLL (Generalized LL) parsing algorithm [8]. GLL handles the full class of context-free grammars, including ambiguous and left-recursive ones, while retaining the structural clarity of recursive descent. For an unambiguous grammar, as Bioscoop’s is, GLL runs in linear time.

The internal front-end is the `bioscoop` macro:

```
1 (defmacro bioscoop [& forms]
2   (let [ast-nodes (mapv form->ast forms)
3         program-ast (vec (concat [:program] ast-nodes))
4         locals      (keys &env)]
5     ` (binding [config/*dynamic-resolution* true]
6       (let [env# (reduce (fn [e# [k# v#]]
7                           (env-put e# k# v#))
8                           (make-env)
9                           ~ (mapv (fn [sym]
10                                   [(str sym) sym]
11                                   locals)))]
12         (run-ast ~program-ast env#))))
```

Listing 3. The bioscoop macro

`form->ast` is a structural transformation from Clojure forms to AST nodes. It handles all surface syntax cases: function calls, symbols, keywords, strings, numbers, booleans, padded graphs, maps, and the `for`, `let`, and `compose` special forms.

C. Multi-Stage Programming via `&env`

A significant design challenge is threading Clojure lexical bindings into the DSL’s runtime environment. Consider:

```
1 (defn n-transition [n offset]
2   (bioscoop
3     (for [i (range n)]
4       [(str "v" i)]
5       (xfade {:transition "fade"
6                :duration 1
7                :offset (+ i offset (* i offset))})
8       [(str "t" (inc i))]))))
```

Listing 4. Parameterized iteration with lexical scope

Here `n` and `offset` are Clojure locals, not DSL bindings. The `bioscoop` macro must make them visible to the DSL evaluator at runtime.

The solution uses multi-stage programming [9]: (`keys &env`) captures the names of all local bindings at macro *expansion time*, as a sequence of symbols. The macro then generates code that, at *runtime*, evaluates those symbols to

their current values and injects them into the DSL environment:

```

1 ; At expansion time, for locals [n offset]:
2 ; (keys &env) = (n offset)
3 ; (mapv (fn [sym] [(str sym) sym]) locals)
4 ; produces the literal: [{"n" n} ["offset" offset]]
5 ;
6 ; At runtime, n=10 and offset=6, so:
7 ; [{"n" 10} ["offset" 6]]
8 ; is injected into the DSL env

```

Listing 5. The locals injection mechanism

The key asymmetry is that `(str sym)` converts the symbol to its string name at expansion time — the only moment the name is available as a symbol — while `sym` remains unevaluated so that the generated code reads its value from the live call frame at runtime. This is a two-phase computation: names extracted at compile time, values read at runtime.

IV. THE INTERMEDIATE REPRESENTATION

A. Algebraic Structure

The IR is defined by three record types:

```

1 (defrecord Filter [name args
2                   input-labels output-labels
3 ])
4 (defrecord FilterChain [filters])
5 (defrecord FilterGraph [chains])

```

Listing 6. IR record definitions

These define a context-free grammar over Clojure records:

$$\begin{aligned}
 \text{FilterGraph} &::= [\text{FilterChain}^*] \\
 \text{FilterChain} &::= [\text{Filter}^+] \\
 \text{Filter} &::= \text{name} \times \text{args} \times \text{labels}^* \times \text{labels}^*
 \end{aligned}$$

Well-formedness is enforced by construction: `make-filtergraph` accepts only vectors of `FilterChain` records, `make-filterchain` accepts only vectors of `Filter` records. Invalid structures cannot be constructed through the public API.

B. Isomorphism with the Target Grammar

The IR grammar and the FFMpeg filtergraph BNF grammar are in bijection:

This isomorphism has a significant architectural consequence: the back-end `to-ffmpeg` is a *serializer* rather than a compiler pass. It traverses a structure already in the shape of the target and emits delimiters:

```

1 (extend-protocol Renderable
2   Filter
3   (to-ffmpeg [f]
4     (let [in (str/join " "
5                       (map #(str "[" % "]")
6                             (:input-labels f)))
7           out (str/join " "
8                       (map #(str "[" % "]")
9                             (:output-labels f)))
10          args (when (:args f)

```

TABLE I
IR TO FFMPEG GRAMMAR CORRESPONDENCE

IR	FFmpeg BNF
FilterGraph	filtergraph
FilterChain	filterchain
Filter	filter
:name	filter-name
:args	filter-arguments
:input-labels	input-linklabels
:output-labels	output-linklabels
; between chains	filterchain separator
, between filters	filter separator

```

11 (str "=" (str/join ":"
12                (map (fn [[k v]]
13                      (str (name k) "=" v))
14                          (:args f))))))
15 (str in (:name f) args out)))
16 FilterChain
17 (to-ffmpeg [fc]
18   (str/join "," (map to-ffmpeg (:filters fc))))
19 FilterGraph
20 (to-ffmpeg [fg]
21   (str/join ";" (map to-ffmpeg (:chains fg))))

```

Listing 7. to-ffmpeg as serializer

No information is added or removed. The serializer is a homomorphism from the IR algebra to strings under concatenation.

C. Round-Trip Correctness

The isomorphism also enables a bidirectional parser: `ffmpeg-parser` parses native FFMpeg filtergraph strings into the same record structure that the DSL compiler produces. We state the round-trip property formally and give a proof sketch by structural induction.

Let $\mathcal{S}$ denote the set of well-formed FFMpeg filtergraph strings, $\mathcal{I}$ the set of well-formed IR values, $\sigma : \mathcal{I} \rightarrow \mathcal{S}$ the serializer `to-ffmpeg`, and $\pi : \mathcal{S} \rightarrow \mathcal{I}$ the parser `ffmpeg-parser`.

Theorem IV.1 (Round-trip correctness). *For all $v \in \mathcal{I}$, $\pi(\sigma(v)) = v$.*

Proof sketch. By structural induction on the IR grammar.

Base case: Filter. Let $f = \langle n, a, \ell_i, \ell_o \rangle$ where n is the filter name, a the argument map, ℓ_i the input labels, and ℓ_o the output labels. The serializer produces:

$$\sigma(f) = \underbrace{[\ell_{i_1}] \cdots [\ell_{i_k}]}_{\text{input labels}} \underbrace{n = a_1 : a_2 : \cdots}_{\text{args}} \underbrace{[\ell_{o_1}] \cdots [\ell_{o_m}]}_{\text{output labels}}$$

The FFMpeg grammar rule `filter = linklabel* filter-spec linklabel*` parses this string unambiguously, recovering n from `filter-name`, a from `filter-arguments`, ℓ_i from leading `linklabels`, and ℓ_o from trailing `linklabels`. Since labels are stored as first-class fields (not metadata), equality holds: $\pi(\sigma(f)) = f$.

Inductive case: FilterChain. Let $c = [f_1, \dots, f_n]$ where each f_j is a `Filter`. The serializer produces $\sigma(f_1), \sigma(f_2), \dots, \sigma(f_n)$. The FFMpeg grammar rule

`filterchain = filter ("," filter)* partitions` this string at each top-level comma (commas do not appear in filter names, argument keys, or labels), yielding n substrings. By the base case, $\pi(\sigma(f_j)) = f_j$ for each j . Therefore $\pi(\sigma(c)) = [f_1, \dots, f_n] = c$.

Inductive case: FilterGraph. Let $g = [c_1, \dots, c_m]$ where each c_k is a `FilterChain`. The serializer produces $\sigma(c_1) ; \sigma(c_2) ; \dots ; \sigma(c_m)$. The FFMpeg grammar rule `filtergraph = filterchain (";" filterchain)* partitions` this string at each top-level semicolon (semicolons do not appear within filterchains). By the inductive case, $\pi(\sigma(c_k)) = c_k$ for each k . Therefore $\pi(\sigma(g)) = [c_1, \dots, c_m] = g$. $\square$

The converse — $\sigma(\pi(s)) = s$ for all $s \in \mathcal{S}$ — does not hold in general because the FFMpeg serialization format admits multiple syntactic representations of the same filtergraph (equivalent argument orderings, optional whitespace). Bioscoop normalizes these to a canonical form during parsing. The round-trip that is guaranteed is $\text{IR} \rightarrow \text{string} \rightarrow \text{IR}$, which is the direction that matters for the compiler's correctness.

D. Labels as First-Class Fields

An earlier version of Bioscoop stored labels as Clojure metadata on `Filter` records. This was architecturally incorrect: Clojure's specification states that metadata does not affect equality, so two filters with different labels were equal. This violated the IR grammar, where `[in]scale` and `scale` are distinct constructs.

The current design stores labels as explicit fields. Two filters with different labels are correctly unequal. The Sinkable protocol, which existed solely to manage the metadata workaround, is eliminated. The IR now enforces the grammar's semantics through the type system.

V. THE EVALUATOR

A. Definitional Interpreter in Environment-Passing Style

`transform-ast` is a definitional interpreter [10] written in environment-passing style. Each `defmethod` gives the meaning of one syntactic form by structural recursion over the parse tree:

```
1 (defmulti transform-ast*
2   (fn [node env] (first node)))
3
4 (defn transform-ast [node env]
5   (let [result (transform-ast* node env)]
6     (tap> {:node-type (first node)
7            :node      node
8            :result     result
9            :ffmpeg    (when (instance? FilterGraph
10                          result)
11                          (to-ffmpeg result))})
12   result))
```

Listing 8. The evaluator entry point

The evaluator is:

- *Single-pass*: one recursive descent over the AST
- *Eagerly evaluated*: applicative-order evaluation

- *Lexically scoped*: the environment is a chain of frames linked by parent pointers
- *Total*: every well-formed input produces a `FilterGraph`, possibly empty

The tap integration makes every node's input, output, and intermediate filtergraph string observable without instrumentation. Callers register tap subscribers to capture trace data; no subscribers means no overhead.

B. Environment Model

The environment is a Clojure map extended with a parent pointer:

```
1 (defn make-env
2   ([ ] {:errors (atom [ ])}))
3 ([parent] (assoc {:errors (atom [ ])} :parent
4                  parent)))
5
6 (defn env-get [env sym]
7   (if (contains? env sym)
8       (get env sym)
9       (when-let [parent (:parent env)]
10         (env-get parent sym))))
11
12 (defn env-put [env sym val]
13   (assoc env sym val))
```

Listing 9. Environment operations

`contains?` rather than `if-let` is used to correctly handle bindings to falsy values. New scopes are created with `(make-env parent)`, establishing the parent chain for lexical scope traversal.

C. The for Construct

The `for` construct is the most significant addition to the filtergraph language. It iterates over any seqable Clojure value and produces a `FilterGraph` by merging the results of evaluating the body expression for each iteration value:

```
1 (defmethod transform-ast* :for-binding
2   [[_ [sym-name] range-node & body] env]
3   (let [xs (transform-ast range-node env)]
4     (apply compose-filtergraphs
5            (for [x xs
6                  :let [loop-env (env-put env sym-name x)]]
7              (->> body
8                  (mapv #(transform-ast % loop-env))
9                  (mapv #(promote-to-filtergraph % loop
10                        -env))
11                  (apply compose-filtergraphs))))))
```

Listing 10. for-binding evaluation

The range expression is evaluated against the current environment, so it can reference DSL bindings and Clojure locals injected via `&env`. All of Clojure's sequence-producing functions — `range`, `map`, `filter`, `iterate` — are available as range expressions.

The `for` construct also appears in label position, where it generates a sequence of label strings rather than a filtergraph:

```
1 ; Generates input labels ["b0c" "b1c" ... "b(n-1)c"]
2 ; for the stacking filter:
3 [[(for [i (range n)] (str "b" i "c"))] stacking]
```

Listing 11. for in label position

This is handled by the `eval-label-content` multi-method, which intercepts `:for-binding` nodes in label position and evaluates them as string sequences rather than filtergraph producers.

D. Chain Flattening

The `chain` construct flattens heterogeneous sequences of `Filter`, `FilterChain`, and single-chain `FilterGraph` values into a single `FilterChain`:

```
1 "chain"
2 (make-filterchain
3   (vec (mapcat #(promote-to-filterchain % env)
4               transformed-args)))
```

Listing 12. chain flattening via `promote-to-filterchain`

`promote-to-filterchain` is a normalization function parameterized by an error handler:

```
1 (defn promote-to-filterchain* [f]
2   (fn [x env]
3     (cond
4       (instance? Filter x)      [x]
5       (instance? FilterChain x) (:filters x)
6       (instance? FilterGraph x)
7         (if (= 1 (count (:chains x)))
8             (:filters (first (:chains x)))
9             (f env x :chain-parallel-filtergraph)))
10      :else (f env x :not-a-filtergraph))))
```

Listing 13. `promote-to-filterchain*` factory

This allows `defgraph`-defined named graphs to be used transparently as chain stages, since a `defgraph` with a linear body produces a single-chain `FilterGraph` that flattens into its constituent filters. Multi-chain filtergraphs — representing parallel structure — are rejected as a type error.

VI. ERROR HANDLING

A. Fail-Soft with Typed Sentinel

Bioscoop targets interactive creative coding at the REPL. In this context, a hard-failure model — where compilation errors throw exceptions and abort evaluation — is counter-productive. Partial results and immediate feedback matter more than strict failure boundaries.

The error discipline is fail-soft with explicit sentinel: all error paths return `(make-filtergraph [])` and accumulate a structured error record in a dynamically-scoped error atom. The first error is printed immediately at the point of accumulation. The atom is inspectable after compilation completes:

```
1 (defn accumulate-error*
2   ([env error]
3    (accumulate-error* (make-filtergraph []) env error))
4   ([return-val env error]
5    (let [info (ex-data error)]
6      (log/warn (if *warn-verbose*
7                  info
```

```
8      (:error-type info))))
9    (swap! (:errors env) conj error)
10   return-val))
```

Listing 14. Error accumulation

The `return-val` parameter, defaulting to an empty `FilterGraph`, allows error handlers to return type-appropriate sentinels. `Filterchain` contexts return `[]` since their results feed `mapcat`:

```
1 (def accumulate-error-fg
2   (partial accumulate-error (make-filtergraph [])))
3
4 (def accumulate-error-fc
5   (partial accumulate-error []))
6
7 (def promote-to-filtergraph
8   (promote-to-filtergraph* accumulate-error-fg))
9
10 (def promote-to-filterchain
11   (promote-to-filterchain* accumulate-error-fc))
```

Listing 15. Partial application for context-appropriate sentinels

B. Invariant: *transform-ast* Always Returns *FilterGraph*

The central invariant of the evaluator is that `transform-ast`, when called in filter-producing position, always returns a `FilterGraph`. Error paths return an empty `FilterGraph` rather than a map, an exception, or `nil`. This eliminates defensive type-checking throughout the evaluator: no method needs to guard against receiving a non-`FilterGraph` from a recursive call.

The invariant is established at the program boundary by `promote-to-filtergraph`, which is the only function that transitions from the untyped world of evaluated expressions to the typed world of IR values. Every method that consumes a recursive result calls `promote-to-filtergraph` before working with it.

VII. SPEC-DRIVEN VALIDATION

Filter parameters are validated against `clojure.spec` [11] specifications before construction. Each filter has a corresponding spec defining the valid types and ranges of its parameters:

```
1 (s/def ::decay (s/double-in :min 0 :max 1))
2 (s/def ::planes (s/int-in 0 16))
3 (s/def ::lagfun (s/keys :opt-un [::decay ::planes
4                                 ]))
```

Listing 16. Example spec for the `lagfun` filter

Parameter maps use unqualified keys in DSL source. `spec-aware-namespace-map` resolves these to their correct namespace-qualified forms by consulting the spec registry:

```
1 (defn spec-aware-namespace-map [spec-keyword m]
2   (let [spec-form (s/form spec-keyword)
3         key-mapping
4           (into {}
5             (map (fn [qk]
6                    [(keyword (name qk)) qk])
7                  (all-keys spec-form)))]
```



```

8 (into {}
9   (map (fn [[k v]]
10         [(get key-mapping k k)
11          (coerce-numeric v)]))
12   m)))

```

Listing 17. Namespace resolution via spec introspection

The `coerce-numeric` function converts Clojure Ratio values (produced by integer division) to Double, ensuring compatibility with specs defined via `s/double-in`. This is a latent type error that expression arguments expose: `(/ 1 3)` produces a Ratio in Clojure, not a Double.

VIII. DISCUSSION

A. Applicability of the Isomorphic IR Approach

The absence of a lowering problem in Bioscoop is a consequence of domain analysis, not a limitation of ambition. FFmpeg filtergraphs are already a high-level, compositional, declarative language. The source language and target language are at the same level of abstraction. There is no semantic gap to bridge, only a syntactic and ergonomic one.

This situation is not universal. A compiler from a higher-level language to machine code must lower closures, garbage collection, type erasure, and many other abstractions that have no counterpart in the target. The isomorphic IR approach is appropriate when:

- 1) The target language is already structured and compositional
- 2) The source language adds abstraction mechanisms rather than new semantic concepts
- 3) The primary function of the compiler is elaboration — name resolution, validation, iteration expansion — rather than semantic transformation

FFmpeg filtergraphs satisfy all three conditions. Configuration languages, build system descriptors, query languages, and other declarative domain notations typically do as well.

B. AST Convergence as a Design Pattern

AST convergence is a design decision, not a free consequence of homoiconicity. Without deliberate maintenance, the two front-ends will diverge as the language evolves: new constructs will be added to the macro path but not the grammar, or vice versa, and the behavioral equivalence guarantee will silently erode.

The mechanism that enforces convergence is the shared AST node type set. Both front-ends must produce nodes from the same enumerated set. When a new construct is added — `for`, for instance — it must be added to both the grammar and `form->ast` simultaneously. The test suite enforces this by running identical assertions against both compilation paths:

```

1 (deftest macro-string-convergence
2   (is (= (to-ffmpeg
3         (compile-dsl
4          "(chain (scale 1920 1080) (eq))"))
5         (to-ffmpeg

```

```

6 (bioscoop
7   (chain (scale 1920 1080) (eq))))))

```

Listing 18. Convergence test pattern

C. Single-Pass Architecture

The evaluator is single-pass in the sense that one recursive descent over the AST produces the final IR. There are no analysis phases, no optimization passes, and no separate code generation step. The `:program` method processes graph definitions before regular expressions via `doseq`, establishing an ordering constraint, but this does not constitute a second pass: no new representation is produced and the same environment is used throughout.

This characterization aligns with the classical definition: a compiler pass is a complete traversal of the program representation that produces a new representation. Since Bioscoop uses one representation throughout, it has one pass regardless of the traversal order of individual nodes.

D. Dual Resolution Strategies

Bioscoop targets two distinct execution environments: the JVM Clojure runtime and GraalVM native image. The `resolve-function` multimethod selects between two resolution strategies via the `*dynamic-resolution*` dynamic var.

The JVM path uses runtime reflection via `ns-resolve`. This affords capabilities unique to the JVM’s open-world execution model: vars defined after the program started, graphs interned via `defgraph` in arbitrary namespaces, and fully qualified symbol resolution across dynamically loaded code. User-defined filtergraph functions are resolved by searching the current namespace and its transitive dependencies at runtime, enabling interactive development patterns where graphs are defined and redefined at the REPL without restarting the system.

The GraalVM native image path uses a static lookup table populated at compile time from the public vars of `bioscoop.built-in` and `clojure.core`. GraalVM’s closed-world compilation model restricts dynamic class loading and requires explicit reflection configuration. The static table is the correct response to these constraints: it is fast, allocation-free, and requires no reflection at runtime.

These are not two implementations of the same mechanism but two distinct strategies, each maximizing the affordances of its execution environment. The `*dynamic-resolution*` dynamic var is the abstraction boundary between them, selected automatically by the macro path and configurable for programmatic use.

E. Limitations

The current system has one notable limitation.

Label arity at compose boundaries: The `for` construct can generate n parallel filtergraph chains, each with computed output labels. But a subsequent `compose` stage that collects all n outputs as inputs to a single filter — such as `concat` or `xstack` with n inputs — must still list those

labels explicitly. A splice operator for label sequences at compose boundaries would close this gap.

IX. RELATED WORK

A. Language Workbenches

JetBrains MPS [12] and Xtext [13] provide tooling for multi-modal DSL development, typically maintaining separate grammars for different surface syntaxes. Neither targets the problem of shared intermediate representation between modalities.

B. Nanopass Framework

The Nanopass Framework [14] for Scheme decomposes compilation into many small, explicitly named passes, each operating on a distinct intermediate language. This approach maximizes comprehensibility of individual passes at the cost of managing many intermediate representations. Bioscoop’s single-representation approach is at the opposite end of this design spectrum, appropriate because the source and target languages are at the same level of abstraction.

C. Typed DSLs

Typed embeddings of DSLs in functional languages [15] use the host type system to enforce DSL well-formedness statically. Bioscoop uses runtime validation via `clojure.spec` rather than static types, trading compile-time guarantees for flexibility and REPL interactivity.

X. CONCLUSION

We have described Bioscoop, a domain-specific language compiler for FFMpeg filtergraph composition. Its primary contributions are:

- *AST convergence*: a deliberately maintained architectural property ensuring that two front-ends produce identical ASTs before evaluation, guaranteeing semantic equivalence by construction
- *Isomorphic IR*: an intermediate representation whose grammar is in bijection with the target language’s grammar, reducing the back-end to a serializer
- *First-class iteration*: a `for` construct that generates parallel filtergraph structures, the first abstraction mechanism of its kind in FFMpeg tooling
- *Multi-stage locals injection*: a mechanism using `&env` to thread Clojure lexical bindings into the DSL runtime evaluator
- *Dual resolution strategies*: a design that maximizes the affordances of two distinct execution environments — runtime reflection on the JVM and static lookup in GraalVM native image — rather than reducing both to a common denominator
- *Fail-soft error discipline*: appropriate for interactive creative coding, where partial results are more useful than hard failures

The system is available as open-source software. Its most practically significant property is the `for` construct, which

closes the expressiveness gap between FFMpeg’s static filtergraph syntax and the dynamic, parameterized pipeline descriptions that users of complex filtergraphs actually need.

ACKNOWLEDGMENT

The author thanks the Clojure community for their contributions to the ecosystem that made this work possible, and the FFMpeg project for building a framework expressive enough to warrant a compiler.

REFERENCES

- [1] K. Zhang, *ffmpeg-python: Python bindings for FFMpeg*. [Online]. Available: <https://github.com/kkroening/ffmpeg-python>
- [2] D. Schaffe, *fluent-ffmpeg: A fluent API for FFMpeg*. [Online]. Available: <https://github.com/fluent-ffmpeg/node-fluent-ffmpeg>
- [3] Z. Oumarou, *MoviePy: Video editing with Python*. [Online]. Available: <https://zulko.github.io/moviepy/>
- [4] P. Hudak, “Building domain-specific embedded languages,” *ACM Computing Surveys*, vol. 28, no. 4es, p. 196, 1996.
- [5] M. Flatt and PLT, “Reference: Racket,” Technical Report PLT-TR-2010-1, PLT Inc., 2010. [Online]. Available: <https://racket-lang.org/tr1/>
- [6] R. Hickey, “The Clojure programming language,” in *Proceedings of the 2008 Symposium on Dynamic Languages*, 2008, p. 1.
- [7] M. Engelberg, *Instaparse: What if context-free grammars were as easy to use as regular expressions?* [Online]. Available: <https://github.com/Engelberg/instaparse>
- [8] E. Scott and A. Johnstone, “GLL parsing,” *Electronic Notes in Theoretical Computer Science*, vol. 253, no. 7, pp. 177–189, 2010.
- [9] W. Taha and T. Sheard, “MetaML and multi-stage programming with explicit annotations,” *Theoretical Computer Science*, vol. 248, no. 1–2, pp. 211–242, 2000.
- [10] J. C. Reynolds, “Definitional interpreters for higher-order programming languages,” *Higher-Order and Symbolic Computation*, vol. 11, no. 4, pp. 363–397, 1998.
- [11] R. Hickey, “clojure.spec,” [Online]. Available: <https://clojure.org/guides/spec>
- [12] S. Dmitriev, “Language oriented programming: The next programming paradigm,” *JetBrains onBoard*, vol. 1, no. 1, 2004.
- [13] S. Eysholdt and H. Behrens, “Xtext: implement your language faster than the quick and dirty way,” in *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications Companion*, 2010, pp. 307–309.
- [14] A. Keep and R. K. Dybvig, “A nanopass framework for commercial compiler development,” in *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*, 2013, pp. 343–350.
- [15] J. Carette, O. Kiselyov, and C. Shan, “Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages,” *Journal of Functional Programming*, vol. 19, no. 5, pp. 509–543, 2009.